4 Modello di sicurezza in Java 1.2

Il modello della sicurezza del JDK 1.2 (Java 2) ha la possibilità di creare tante tipologie di permessi a seconda delle esigenze, potendo controllare nei minimi dettagli tutti gli aspetti coinvolti. Il tutto parte dal lato client dove si può specificare, in base alla provenienza dell'applet, il tipo di operazioni che esso potrà eseguire in locale, e su quali risorse. In figura è riportata l'attuale configurazione del sistema.

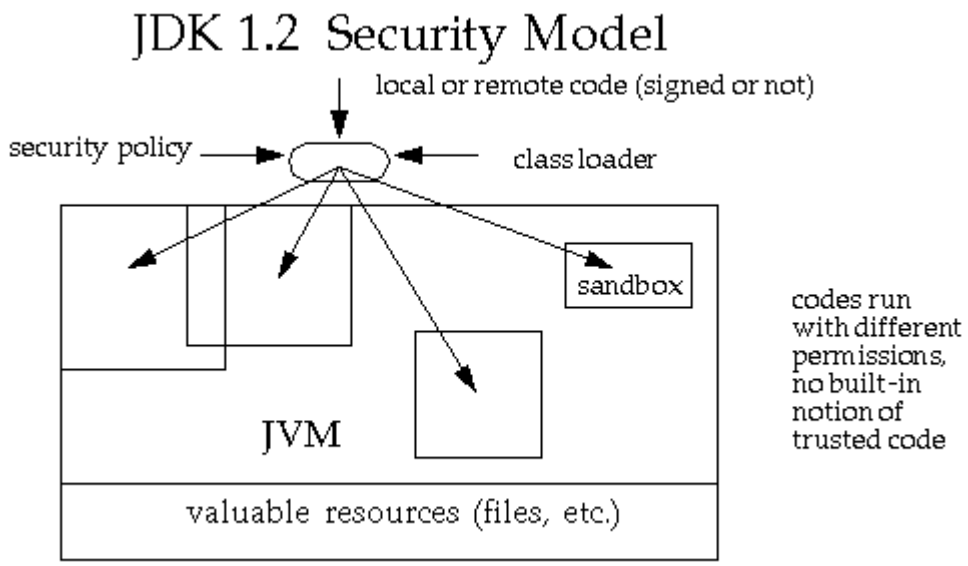


Fig. 4.1 Il modello di sicurezza di Java 2.Ogni codice viene classificato in base alla coppia di valori origine/firma ed eseguito nel rispetto dei privilegi concessi dalla politica

Al fianco del SM adesso troviamo l'Access Controller (AC), che offre la possibilità di personalizzare gli accessi, esso permettere ad un codice di fare gradualmente dei passi fuori dai vincoli della sandbox specificando gli adeguati permessi. Il SM continua ad essere presente per mantenere la compatibilità all'indietro con le applicazioni basate sul JDK 1.1; in questo caso, però, esso esegue solo una interfaccia per l'AC che effettua in background tutti i controlli.

La possibilità di modificare la gestione dei permessi consente di cambiare completamente la politica della sicurezza semplicemente editando un file di configurazione (anche in runtime), cosa possibile in precedenza solo sostituendo completamente il SM (operazione non facile e non sempre possibile).

Il funzionamento dell'AC si basa sui seguenti concetti:

Provenienza del codice: specifica da dove il codice è stato scaricato se si tratta di un host remoto, o caricato se proviene dal file system locale;

Permesso: una serie di permessi disponibili che verranno associati alle singole operazioni eseguibili sulla macchina;

Politica di sicurezza: un insieme di associazioni che specificano le operazioni che possono essere eseguite e da chi;

Dominio di protezione: un insieme di privilegi (o permessi) associati ad un insieme di programmi Java (applet o applicazioni) con la stessa provenienza, e firmati elettronicamente da utenti appartenenti allo stesso gruppo.

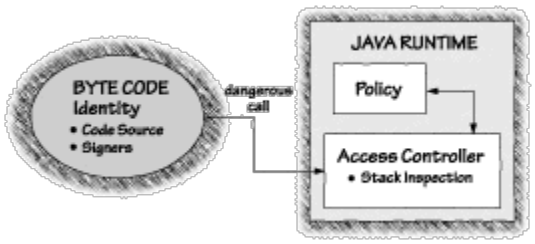


Fig. 4.2 Il codice mobile in Java 2 interagisce con la politica definita dall'utente attraverso l'Access Controller.Il bytecode può eseguire chiamate alle funzionalità potenzialmente pericolose. Quando tali chiamate sono fatte, l'AccessController (nuovo in Java 2) consulta la politica e usa il meccanismo di *stack inspection* per decidere se permettere o respingere una chiamata. Le decisioni si basano sull'identità del codice.

4.1 Permessi

L'elemento base su cui l'AC basa il suo funzionamento è l'oggetto (inteso come entità) *Permesso*, che corrisponde alla classe java.security.Permission. A tale classe sono attribuiti due significati distinti: quando un oggetto Permission è associato ad una classe, esso rappresenta l'insieme di operazioni che sono state garantite alla classe stessa; altrimenti esso consente di sapere se si possiedono i permessi necessari per poter eseguire una certa operazione. Il possedere una classe di permesso di accesso a file, non significa che si possa realmente accedere al file, ma rende noto se è possibile farlo. Una classe Permission ha tre proprietà fondamentali:

Un tipo: il tipo di permesso che si intende specificare (accesso a file o a socket). Le API di Java mettono a disposizione 11 tipi di permessi, che vanno dall'accesso a file (java.io.FilePermission), a permessi legati alla possibilità di accedere a risorse di rete via socket (java.net.SocketPermission), a classi legate a permessi di esecuzione di operazioni su thread e simili (java.lang.Runtimecission);

Un nome: anche se non è obbligatorio e non è stata definita una regola precisa di naming, in genere si cerca di mantenere una relazione piuttosto stretta fra il nome e la sua area di interesse. MyFilePermission piuttosto che MyPermissioXYZ nel caso di permessi su file;

Una serie di azioni: in funzione della tipologia di permesso, è specificata una serie di operazioni che è possibile eseguire. Ad esempio, creando ex-novo un oggetto FilePermission possiamo pensare di fornire azioni come scrittura o lettura del file, mentre in altri casi tali azioni non avrebbero senso. Da notare che la classe Permission è astratta, e che quindi non è utilizzabile direttamente: dal punto di vista del programmatore generalmente essa serve principalmente per la definizione di permessi particolari. Dispone di alcuni metodi di servizio che però non sono utilizzati nella programmazione generica.
La classe BasicPermission può essere utile in fase di sviluppo, dato che può essere utilizzata come mattone elementare da cui partire: non dispone di azioni e rappresenta in pratica un permesso binario (si/no) molto

simile a quello del modello precedente.

4.2 La Politica

In pratica, la politica è impostata in un file di configurazione che è caricato durante il setup della VM. In queste politiche, una chiave pubblica (di solito una stringa di molti bit) è rappresentata attraverso un alias. L'alias è il nome di un firmatario rappresentato come una stringa. Per esempio, l'alias "self" rappresenta la tua propria chiave privata. Meccanismi primitivi sono inclusi per creare e importare chiavi pubbliche e certificati nel sistema di Java.
Esiste per default un singolo file della politica del sistema e un singolo file della politica definita dell'utente. Il primo è localizzato in

{java.home}/lib/security/java.policy (Solaris)

{java.home}\lib\security\java.policy (Windows)

dove java.home è una proprietà del sistema che specifica la directory in cui JDK è installato. Il secondo, in

{user.home}/.java.policy (Solaris)

{user.home}\.java.policy (Windows)

dove, user.home è una proprietà del sistema che specifica la home-directory dell'utente. Se nessuna politica è trovata, è utilizzata la politica di default, quella rappresentata dal modello sandbox.
L'identità del codice viene confrontata con tutte le entry dell'oggetto Policy per determinare i permessi da assegnare al codice. Si ottiene una corrispondenza tra l'identità del codice e una entry dell'oggetto Policy se entrambi i valori, origine e firma, sono soddisfatti. Questo significa che, in termini di firma, l'URL che definisce l'origine del codice deve essere identica ad uno dei valori di una entry dell'oggetto Policy. Mentre, in termini di firma, la chiave pubblica (associata alla chiave privata legata alla firma del codice) deve essere uguale alla chiave pubblica di un firmatario nella politica del sistema.
Un codice può essere firmato con una firma multipla. Nel caso in cui le firme, che il codice porta con se, hanno corrispondenze multiple con le entry nella politica, tutte le entry sono applicate in modo additivo. Questo significa che al codice sono assegnati tutti i permessi, dati dall'unione dei singoli corrispondenti permessi.

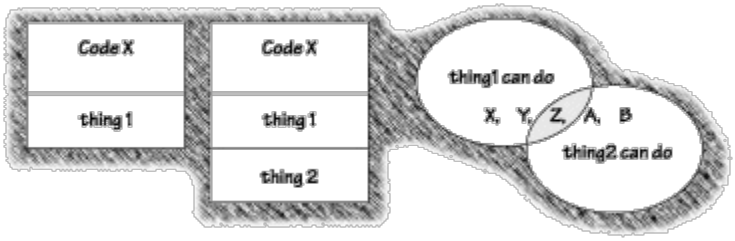


Fig. 4.3 Il pericolo di una politica additiva

Consideriamo il programma X mostrato in figura 4.3. Nel primo caso, X è firmato da thing1. Nell'altro caso, il codice è firmato da thing1 e thing2. Nel secondo caso, la politica di entrambi thing1 e thing2 è applicata al codice (significa che esso ha più opportunità di fare attività pericolose). Un amministratore può dimenticare di valutare che cosa accadrebbe quando un codice è firmato da multiple chiavi.

4.3 Domini di Protezione

Un dominio di protezione rappresenta l'insieme di tutti i permessi che una certa classe (programma client sulla JVM) può eseguire. Un dominio è rappresentato materialmente dalla classe java.security.ProtectionDomain, il cui costruttore è

public ProtectionDomain(CodeSource cs, Permission p)

Associare un ProtectionDomain ad una certa classe, permette di specificarne il sito di provenienza (specificato dal nome del sorgente cs), la firma elettronica (sempre specificata in cs), ed il set di regole (date da p) alle quali deve sottostare. Java runtime mantiene una associazione tra codice e domini di protezione.

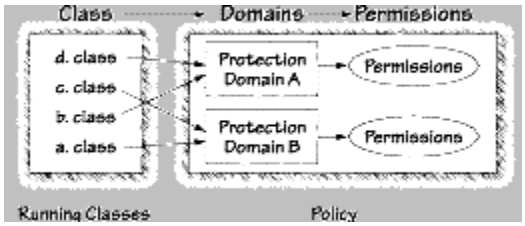


Fig. 4.4 Le classi vengono associate a domini di protezione e subito dopo ai permessi. La politica è definita in termini di domini di protezione.

La politica di sicurezza specifica quali domini di protezione dovrebbero essere creati e a quali domini dovrebbero essere assegnati i permessi. Esiste un dominio di protezione speciale: il dominio del sistema (system protection domain). Quest'ultimo include tutto il codice del sistema caricato con il Primordial Class Loader. Questo include le classi nella CLASSPATH. Al dominio del sistema sono forniti privilegi speciali.

4.4 Access Control e Stack Inspection

La classe java.security.AccessController non è accessibile direttamente (dato che il costruttore è protetto) e possiede una serie di metodi statici molto utili all'applicazione client per determinare a quali restrizioni è soggetta. In tal senso il metodo fondamentale, il checkPermission(), prende come parametro un permesso e determina se la classe è autorizzata a procedere.
Quando un'applicazione Java ha bisogno di eseguire un codice non-fidato, le librerie del sistema Java necessitano di qualche modalità per distinguere le chiamate originate da un codice fidato e quelle originate da uno non-fidato. Chiaramente, le chiamate originate dal codice non-fidato devono essere ristrette per prevenire attività ostili, mentre le altre devono godere di maggiore libertà.
La soluzione per implementare un tale sistema è quella di esaminare il runtime stack. Ogni thread in esecuzione ha il suo proprio runtime stack. Le decisioni di sicurezza vengono prese riferendosi a questo controllo chiamato STACK INSPECTION (Fig. 4.5).

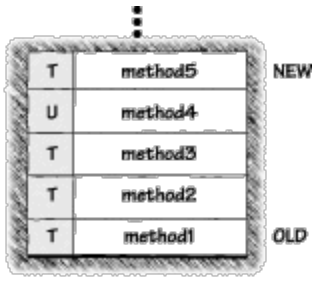


Fig. 4.5 Ogni programma Java include un runtime stack che mantiene traccia delle chiamate ai metodi. Lo scopo dello stack è di mantenere traccia di quel metodo che chiama successivamente quell'altro metodo per essere abile a ritornare alla giusta locazione del programma quando un metodo invocato termina il suo lavoro. Lo stack cresce e si restringe durante le tipiche operazioni del programma. Java 2 ispeziona nell'ordine le chiamate nello stack per prendere le decisioni di accesso. In questo esempio, ogni stack frame include una chiamata ad un metodo ed una etichetta di fiducia (T per trusted e U per untrusted).

Vediamo in dettaglio i tre scopi della classe Java.security.AccessController :

1. Se permettere o negare l'accesso a una risorsa, utilizzando la politica corrente.
2. Marcare il codice come "privileged", così da influenzare la determinazione degli accessi successivi.
3. Ottenere un'"istantanea" (snapshot) del contesto delle chiamate correnti, cosicché le decisioni sul controllo d'accesso possono essere prese rispetto ad un contesto salvato.

Ogni codice che controlla l'accesso alle risorse del sistema dovrebbe invocare i metodi di AccessController se desidera usare il modello di sicurezza specificato e gli algoritmi utilizzati da questi metodi. Se, diversamente, l'applicazione desidera differire dal modello di sicurezza per quello della SecurityManager installato runtime, allora dovrebbe, invece, invocare i corrispondenti metodi della classe SecurityManager.

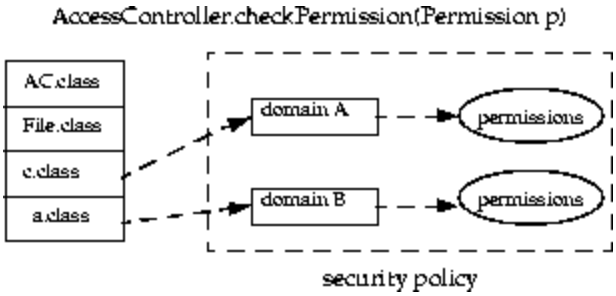
Un tipico modo di invocare il controllo d'accesso:

FilePermission perm = new FilePermission("path/file", "read");

AccessController.checkPermission(perm);

Il metodo AccessController.checkPermission esamina il corrente contesto dell'esecuzione e prende la giusta decisione se permettere o negare l'accesso. Se tutto va bene, il controllo ritorna silenziosamente. Altrimenti, una AccessControlException (una sottoclasse di java.lang.SecurityException) è lanciata.

Supponiamo di dover verificare un controllo d'accesso in un thread che ha una catena di multipli chiamanti, lo scenario viene illustrato dalla seguente figura:



Quando il metodo AccessController.checkPermission è invocato dal chiamante più recente, viene eseguito il seguente algoritmo:

Se qualche chiamante nella catena delle chiamate non ha il permesso richiesto,
lancia un AccessControllerException, a meno che la seguente cosa è vera:
un chiamante il cui dominio concede il permesso è stato impostato come "privileged" e tutte le parti
successivamente chiamate (direttamente o indirettamente) hanno tutte il permesso citato

Ogni volta che è richiesto un controllo del permesso, lo stato del thread è esaminato per decidere.

Supponiamo che il corrente thread attraversa *m* chiamanti, nell'ordine dal *chiamante_1 al chiamante_2, ... al chiamante_m*. In seguito il *chiamante_m* invoca il metodo checkPermission. L'algoritmo elementare di checkPermission è:

```
i = m;

while (i > 0) {

    if ( il dominio del chiamante i non ha i permessi )

        throw AccessControlException

    else if (il chiamante i viene marcato come privileged)

        return;

    i = i - 1;

};
```

L'algoritmo presentato ha la caratteristica fondamentale di non far guadagnare permessi ad un metodo se quest'ultimo chiama o viene chiamato da un metodo che ha privilegi maggiori. E' fondamentale che l'algoritmo sia stato progettato per rispettare questa regola quando un thread attraversa multipli domini con differenti permessi, altrimenti ci sarebbe stato un piccolo difetto nell'architettura di sicurezza facilmente utilizzabile per implementare applet ostili.

4.4.1 Blocchi Privileged

Un nuovo metodo di AccessController è il doPrivileged(). Con Questo metodo la VM può efficientemente garantire che i privilegi sono revocati una volta che il metodo è stato completato.

Ecco come viene usato il nuovo metodo:

```
metodogenerico() {
    < codice innocuo >
    AccessController.doPrivileged( new PrivilegedAction() {
        public Object run() {
            < inserire qui il codice pericoloso >
            return null;
        });
        < altro codice innocuo >
}
```

Ironicamente, una delle regole, per gli sviluppatori per ottenere un codice più sicuro, è di evitare l'uso delle classi interne (inner class). Ma se si vuole includere blocchi privilegiati nel codice in Java 2, la SUN stessa ne incoraggia l'uso. In aggiunta esiste il problema della verbosità, cioè della difficoltà a leggere e capire il codice.